

Functions in C

Matteo Fasiolo

Entry Point

```
#include <stdio.h>

int main(void)
{
    // Program always starts HERE!!
    printf("My name is Matteo Fasiolo. \n");
    // ... other stuff
    return 0;
}
```

After compiling, where does the program start running?

main function is **the entry point of your program**.

- when a normal C program is run, execution of your code begins in main.

All C (and C++) programs must have a `main` function.

Case Study: Hey Jude

Hey Jude 1.0

```
#include <stdio.h>
int main(void){

    printf("Hey Jude, don't make it bad.\n");
    printf("Take a sad song and make it better.\n");
    printf("Remember to let her into your heart,\n");
    printf("Then you can start to make it better.\n");

    printf("\n");

    printf("Hey Jude, don't be afraid.\n");
    printf("You were made to go out and get her.\n");
    printf("The minute you let her under your skin,\n");
    printf("Then you begin to make it better.\n");

    return 0;
}
```

Writing Functions

You can write all your code in `main`

- it will work, but it is **very messy!**
- It is like writing an essay in only one paragraph.

We want to split the program into pieces.

Functions are basic units of your program.

- They split your program into meaningful pieces.
- Similar to “sections” in an essay.

Hey Jude 2.0

Consider the following code:

```
#include <stdio.h>

void verse1(){
    printf("Hey Jude, don't make it bad.\n");
    printf("Take a sad song and make it better.\n");
    printf("Remember to let her into your heart,\n");
    printf("Then you can start to make it better.\n");
}

void verse2(){
    printf("Hey Jude, don't be afraid.\n");
    printf("You were made to go out and get her.\n");
    printf("The minute you let her under your skin,\n");
    printf("Then you begin to make it better.\n");
}

\\ ... rest of the code
```

The main function calls the two helper functions:

```
// Code from the previous slide goes here
int main(void){

    printf("Hey Jude by The Beatles \n");

    verse1(); // function call statement

    printf("\n");

    verse2(); // function call statement

    return 0;
}
```

- Above, we wrote two functions: `verse1` and `verse2`.
- The program starts at `main`
- The CPU executes statements sequentially.
 - When `verse1` is **called**, it **enters** the function `verse1` and executes sequentially.
 - Once it finishes `verse1`, it **exits** `verse1` and continues where it left off in `main`.
- It continues to execute statements sequentially in `main`.
 - When `verse2` is **called**, it **enters** the function `verse2` and execute sequentially.
 - Once it finishes executing `verse2`, it **exits** `verse2` and continue where it left off in `main`.
- The CPU reaches the end of `main`, the program stops.

Switch to VS Code

Open `heyjude.c`

Functions

1. What is a function?
2. How to define a function?
 - Data Types in C
3. Function Body
 - Variable Declarations
 - Expressions
4. How to call a function?

1: What is a function?

A mathematical function:

$$f(x) = ax + b.$$

- It receives an **input** x .
- It produces an **output** following a certain rule.
- It incorporates parameters a and b .
- Function in programming is a generalization of this mathematical concept.

Functions in Programming

Functions are individual building blocks of your program that accomplish specific tasks.

- Some functions take *input arguments*.
- Some functions return *an output value*.
- Some functions do not have input or output, but still perform an action, such as printing.

Functions

1. What is a function?
2. **How to define a function?**
 - Data Types in C
3. Function Body
 - Variable Declarations
 - Expressions
4. How to call a function?

2: How to Define a Function?

General template of a function:

```
return_type function_name(  
    input_type_1 parameter_name_1,  
    input_type_2 parameter_name_2,  
    ...  
) {  
    function body  
}
```

1. A function definition starts by indicating the **return type**
 - void means no return value will be produced.
2. Followed by the **function name**.
3. The **parameter list** comes after that, inside (and).
4. The **body of the function** is enclosed by { and }.

You can write your own function and call it from `main()`:

```
#include <stdio.h>

void sayhello(){

    printf("Hello World!\n");

}

int main(void){

    sayhello(); // calling "sayhello" function.

    return 0;

}
```

- Function names should **reflects what the function does**:
 - e.g. sayhello, sort, add, delete, etc.
 - avoid long and ambiguous name.

```
#include <stdio.h>
double calculate_circumference(double radius){
    double circ = 2.0*3.1415926*radius;
    return circ;
}
int main(void){
    // call calculate_circumference
    double res = calculate_circumference(2.0);
    printf("%f\n", res);
    return 0;
}
```

- calculate_circumference takes one input argument radius and **returns** a decimal number.
- It is called by main, who will receives its returned value and prints it.
- But... what is double?

Functions

1. What is a function?
2. How to define a function?
 - **Data Types in C**
3. Function Body
 - Variable Declarations
 - Expressions
4. How to call a function?

2a: Data Types in C

- Some data types are:
 - `int` or `long long`: integers
 - `float` or `double`: floating-point numbers
 - `char`: characters
- Specifying data types tells the compiler:
 - how a value is represented in bits 01011...
 - which operations are appropriate
 - how much storage it typically requires
- On modern PCs (and most smartphones):
 - `int` and `float` occupy 4 bytes of memory.
 - `long long` and `double` occupy 8 bytes of memory.
 - `char` occupies 1 byte of memory.

Both `int` and `long long` represent integers, but:

- `int` has a range of -2147483648 to 2147483647.
- `long long` has a range roughly plus or minus 9 quintillion.

Similarly, `double` and `float` represent floating-point numbers, used to approximate real numbers:

- `float` has about 7 decimal significant decimal digits, and handles magnitudes $(-10^{38}, 10^{38})$.
- `double` has about 15 digits, and handles magnitudes $(-10^{308}, 10^{308})$.

`double` having about 15 digits means that:

- 1 and $1 + 10^{-10}$ produce different `double` numbers.
- But 1 and $1 + 10^{-20}$ produce the same `double` number.

We will use mostly `int` and `double`.

Functions

1. What is a function?
2. How to define a function?
 - Data Types in C
3. **Function Body**
 - Variable Declarations
 - Expressions
4. How to call a function?

3: Function Body

The function body may contain any number of statements.

The convention is

```
... function_name(...){ // START of body
    declaration of variables
    ...

    other statements
} // END of body
```

Example

```
#include <stdio.h>

double calc_gravity(double dist){
    // declare variable m1, m2, G and gravity.
    double gravity;
    double G = 0.00000000006674;
    double m1 = 10.0, m2 = 20.0;

    // compute "gravity" using declared variables.
    gravity = G*m1*m2/dist/dist;
    return gravity;
}

int main(void){
    printf("%f", calc_gravity(0.01));
    return 0;
}
```

Declarations

A variable in C is a placeholder of some value.

The value held by a variable can be changed, not the type.

In C programming language, all variables must be **declared**.

The syntax of declaration is: `data_type variable_name`.

- Uninitialised declaration: `double gravity;`
- Declaration **with initializations**: `double m1 = 1.0;`
- Uninitialised Declaration of multiple variables of the same type: `double m1, m2;`
- Declaration of multiple variables of the same type with initializations: `double m1 = 1.0, m2 = 2.0;`

Once a variable is declared, **the variable is assigned a memory space by the compiler.**

If the variable is uninitialized, **the variable can contain whatever (rubbish) value** that is already at that memory location!

Undefined values can lead to unpredictable behaviors and is a source of error!

```
#include <stdio.h>
int main(void){
    // It will print out some garbage value.
    int a;
    printf("%d\n", a);
    return 0;
}
```

If possible, initialize the variable when it is declared.

```
...  
double calc_gravity(double dist){  
    // declare variable m1, m2, G and gravity.  
    double G = 0.00000000006674;  
    double m1 = 10.0, m2 = 20.0;  
    // initialize gravity as soon as it is declared.  
    double gravity = G*m1*m2/dist/dist;  
    return gravity;  
}  
...
```

If you do not know how to initialize the variable, assign a “default value” (e.g. 0).

So that when you see the value, you know this variable has not been assigned any useful value.

Expressions and Assignments

You can use **variables** and **operators** to construct **expressions**:

- $G*m1*m2/dist/dist$

Each expression has a value:

- The value of $G*m1*m2/dist/dist$ is its outcome.

$m1 = 1.0$, $gravity = G*m1*m2/dist/dist$ are all **assignment expressions**:

- The assigns the value of expression on the RHS to the variable on the LHS.
- **The = sign represents assignment, not equality.**

Functions

1. What is a function?
2. How to define a function?
 - Data Types in C
3. Function Body
 - Variable Declarations
 - Expressions
4. **How to call a function?**

4: Calling a Function

You can call a function **after its definition**.

Call a function by writing its name followed by argument values **in the same order as its parameters**.

For example, suppose a function has three parameters:

- `int function_name(int var1, int var2, int var3)`

You can call it by writing `function_name(7, 4, 5)` where:

- 7 is the input of `var1`, 4 is the input of `var2`, ...
- **Each function call is an expression** whose value is the return value of the function
- A call to a `void` function does not produce any usable value.

```
#include <stdio.h>

double calc_gravity(double m1, double m2, double dist){
    double G = 6.674e-11; // same as  $6.6 \times 10^{-11}$ 
    return G*m1*m2/dist/dist;
}

int main(void){
    double res = calc_gravity(10, 20, 0.01);
    printf("%f\n", res);
    return 0;
}
```

What happens to the input?

```
double square(double x) {  
    x = x * x;  
    return x;  
}  
  
int main(void) {  
    double value = 2.0;  
    double answer = square(value);  
    return 0;  
}
```

In `square(value)`, `value` is the **argument** and `x` is the **parameter**.

C copies the value 2.0 into the local parameter `x`.

The name `x` can be used only inside `square`.

The argument is copied

You can think of the call

```
square(value)
```

as if the function call starts with a new local variable:

```
double x = 2.0;
```

Changing `x` does not change `value`.

C does not give `square` the variable `value`.

It creates the local parameter `x` and copies the value of `value` into it.

`x` is local, it can not be used outside of `square`.

Switch to VS Code

Open `square.c`

Summary

We use functions to split programs into smaller, more specific tasks.

Functions can take input variables and return output value.

The function body includes variables declarations and other statements.

By declaring a variable you tell the compiler:

- how a value is represented in bits 01011...
- which operations are appropriate
- how much storage it typically requires

The arguments of a function are copied.

Variables declared within functions are local.